



Experiment No: 3

Student Name: Manasvi sharma

UID:24BCS10527

Branch: BE.CSE

Section/Group:24BCS_TPP 615 (A)

Semester:5th

Date of Performance: 03/08/2026

Subject Name: Competitive Coding – II.

Subject Code: 24CSP-305

1.(a) LeetCode Problem 69: Sqrt(x)

1. Aim: To design and implement an efficient algorithm to find the square root of a non-negative integer using Binary Search without using any built-in exponent functions.

2. Objective:

- 1.To understand array manipulation techniques.
- 2.To handle carry propagation while performing arithmetic operations on arrays.
- 3.To analyze time and space complexity of the solution.
- 4.To validate the solution using test cases and edge cases.

3./Implementation/Code:

```
class Solution {
public:
    int mySqrt(int x) {
        if (x == 0 || x == 1)
            return x;

        int left = 1, right = x;
        int ans = 0;

        while (left <= right) {
            long long mid = left + (right - left) / 2;

            if (mid * mid == x)
                return mid;
            else if (mid * mid < x) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return ans;
    }
};
```

4. Output

✓ Case 1

✓ Case 2

Input

x =
4

Output

2

Expected

2

5. Learning outcomes (What I have learnt):

1. Learned how to traverse an array from right to left.
2. Understood the concept of carry propagation during addition.
3. Learned how to solve problems involving arrays without converting them into integers.
4. Understood how to handle edge cases such as numbers containing all 9s.
5. Improved problem-solving skills using efficient array manipulation techniques.

1.(b) LeetCode Problem 81: Search in Rotated Sorted Array II

1. **Aim:** To design and implement an efficient algorithm to determine whether a target element exists in a rotated sorted array with possible duplicate values using Binary Search.

2.Objective:

1. To apply sorting and two-pointer techniques for array-based problems.
2. To efficiently identify unique quadruplets without duplicates.
3. To analyze time and space complexity of the solution.
4. To validate the solution using test cases and edge cases

3.Implementation/Code:

```
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0, right = nums.size() - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target)
                return true;

            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            else {
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }

        return false;
    }
};
```

4. Output

✔ Case 1

✔ Case 2

Input

```
nums =  
[2,5,6,0,0,1,2]
```

```
target =  
0
```

Output

```
true
```

Expected

```
true
```

5. Learning outcomes (What I have learnt):

5. Learned how to solve the 4Sum problem using sorting and the two-pointer technique.
6. Understood how to remove duplicate quadruplets efficiently.
7. Learned to optimize a brute-force $O(n^4)$ solution to $O(n^3)$.
8. Understood the importance of using `long long` to prevent integer overflow.
9. Improved understanding of array traversal, sorting, and pointer-based algorithms.

1.(c) LeetCode Problem 154: Find Minimum in Rotated Sorted Array II

1. **Aim:** To design and implement an efficient algorithm to find the minimum element in a rotated sorted array that may contain duplicate values using Binary Search.

2. Objective:

1. To apply greedy algorithms and priority queue techniques.
2. To efficiently calculate maximum team performance using sorting and heaps.
3. To analyse time and space complexity of the solution.
4. To validate the solution using test cases and edge cases.

3. Implementation/Code:

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int left = 0, right = nums.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] < nums[right]) {
                right = mid;
            }
            else if (nums[mid] > nums[right]) {
                left = mid + 1;
            }
            else {
                right--;
            }
        }

        return nums[left];
    }
};
```

4. Output:

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2

Input

```
nums =
[1,3,5]
```

Output

```
1
```

Expected

```
1
```

5. Learning outcomes (What I have learnt):

1. Learned how to solve optimization problems using the Greedy Algorithm.
2. Understood the use of Priority Queue (Min Heap) for maintaining the best k elements.
3. Learned how sorting by efficiency simplifies performance calculation.
4. Improved understanding of STL containers such as vector , pair , sort and priority_queue.
5. Learned to design an efficient solution with $O(n \log n)$ time complexity instead of checking all possible teams.